

Mirage: A State-Consistent Deception Infrastructure Using FUSE, Redis, and Large Language Models

Abhinand K Jayan*, Abhishek Wilson*, Hannah EM*, Hrishekesh PV*, Saju C J†

*Department of Computer Science and Engineering (Cyber Security),
Jyothi Engineering College, Cheruthuruthy, India

†Faculty Advisor, Department of Computer Science and Engineering,
Jyothi Engineering College, Cheruthuruthy, India

Abstract—Modern honeypots face a fundamental trade-off between interaction fidelity and operational scalability. Traditional systems such as Honeyd and Cowrie provide scripted responses that sophisticated adversaries trivially detect through state-consistency probes. Emerging large language model (LLM)-based approaches improve realism but introduce a new failure mode: *state hallucination*, where the model contradicts previously established filesystem or session state after its context window is exceeded. This paper presents Mirage, a cognitive deception infrastructure implemented through the Chronos framework. Chronos separates state management from content generation by coupling a FUSE-based userspace filesystem with atomic Redis transactions for state truth, while delegating content synthesis to LLMs on a generate-once, cache-forever basis. The architecture is organised into six layers: a multi-protocol gateway (SSH/HTTP), a POSIX-compliant FUSE interface, a Redis-backed State Hypervisor enforcing consistency through Lua scripts, a Persona Engine for on-demand content generation, a real-time analysis stack mapping attacker behaviour to the MITRE ATT&CK framework, and a Rust-based Layer 0 for high-speed traffic classification. We validate Mirage across four verification phases covering state atomicity, filesystem operations, intelligence integration, and threat detection. Results demonstrate 8,601 atomic file-creation operations per second, sub-millisecond metadata latency, zero state contradictions over extended sessions, and a 78.6% detection rate against realistic attack simulations comprising 28 commands across five adversarial scenarios.

Index Terms—honeypot, deception technology, FUSE, Redis, large language models, state consistency, MITRE ATT&CK, threat intelligence

I. INTRODUCTION

Honeypots remain a cornerstone of active cyber defence, providing controlled environments that lure adversaries and capture intelligence. Despite decades of development [1], [2], current implementations cluster around two poles with complementary weaknesses.

Low-interaction honeypots (Honeyd [3], Cowrie [4]) respond to commands from static scripts or pre-configured rulesets. Their simplicity enables large-scale deployment but limits engagement depth. An attacker who executes `touch /tmp/pwn && ls /tmp` and observes an empty listing immediately identifies the system as a decoy—the file was never materialised in any persistent store.

LLM-based honeypots aim to solve the depth problem by using large language models to generate contextually appropriate responses. However, LLM context windows (4K–128K

tokens) impose an upper bound on conversational memory. After 50–100 commands a model may report the working directory as `/root` despite an earlier `cd /home/attacker`, a contradiction we term *state hallucination*. The LLM’s responses are independent probability samples rather than reads against a persistent data store, so filesystem state, ownership metadata, and file contents can all diverge unpredictably.

We identify a third gap related to **high-interaction honeypots** (full virtual machines): while they offer perfect realism, each instance requires dedicated compute, introduces non-trivial lateral-movement risk, and scales poorly to concurrent adversarial sessions.

Mirage addresses all three gaps through a single architectural principle: *separate state truth from content generation*. Our contribution is the Chronos framework, which:

- 1) Implements a POSIX-compliant FUSE filesystem backed by atomic Redis transactions, eliminating state inconsistency;
- 2) Delegates content synthesis to LLMs under a generate-once, cache-forever policy, eliminating hallucination by design;
- 3) Integrates real-time attacker profiling via MITRE ATT&CK [9] pattern matching, a threat signature library, and a five-level skill classifier;
- 4) Provides a Rust-based pre-flight analysis layer for high-speed protocol classification and circuit-breaker protection.

The remainder of this paper is organised as follows. Section II surveys related work. Section III details the Chronos architecture. Section IV describes the implementation. Section V presents evaluation results. Section VI discusses limitations and future directions. Section VII concludes.

II. RELATED WORK

A. Low-Interaction Honeypots

Honeyd [3] simulates network topology and services using personality engines and script handlers. Cowrie [4] extends the concept to SSH/Telnet with configurable filesystem snapshots. Both systems share a fundamental limitation: state is either absent or emulated through static lookup tables that cannot handle arbitrary sequences of operations atomically.

B. High-Interaction Honeypots

High-interaction systems such as HoneyBow [5] and Argos [6] run full operating system instances. While they provide perfect filesystem realism, each instance costs \$50–\$200/month in cloud environments, and the risk of attacker escape to the underlying host is non-trivial.

C. LLM-Assisted Deception

Recent research has explored using GPT-class models to generate honeypot responses dynamically [7]. These approaches achieve high conversational realism but treat the LLM as both the state oracle and the response generator. When the context window is exhausted, the model hallucinates prior state, producing contradictions detectable by even moderately skilled adversaries.

D. FUSE-Based Security Systems

FUSE (Filesystem in Userspace) has been applied to ransomware detection [8] and data provenance tracking but, to our knowledge, has not previously been combined with a transactional state store and LLM content generation in a deception context.

Mirage’s contribution lies in the novel combination of FUSE for POSIX interception, Redis [10] for atomic state management, and LLMs strictly for content generation—each component addressing a distinct weakness of prior approaches.

III. SYSTEM ARCHITECTURE

Chronos implements a layered architecture with strict separation of concerns. Fig. 1 illustrates the six-layer design and data flow.

A. Layer 1: Multi-Protocol Gateway

The Gateway layer provides entry points for attacker interaction.

SSH Honeypot. Built on Paramiko [11], the SSH server accepts any credential combination—password or public key—logging each attempt as an audit event. Upon authentication, the server provides a banner mimicking Ubuntu 22.04 LTS₁ and enters an interactive command loop. Each command is recorded and forwarded to the analysis stack.₄

HTTP Honeypot. The HTTP server extends Python’s BaseHTTPRequestHandler to simulate vulnerable web endpoints including /admin, /uploads, and /api. Inline threat detection applies regex-based pattern matching against SQL injection, cross-site scripting (XSS), directory traversal,₈ and command injection payloads. The Server response header is spoofed as Apache/2.4.41 (Ubuntu) to reinforce the deception.₁₃

B. Layer 2: FUSE Interface

The ChronosFUSE class (227 lines) implements the fusepy Operations interface, intercepting POSIX system calls at the kernel–userspace boundary. Table I shows the mapping between POSIX syscalls and Chronos actions.

Path resolution is recursive: for /etc/passwd, the resolver starts at root inode 1, performs ZSCORE fs:dir:1

TABLE I
POSIX SYSCALL TO CHRONOS ACTION MAPPING

Syscall	FUSE Method	Chronos Action
getattr	getattr	HGETALL fs:inode:<id>
readdir	readdir	ZRANGE fs:dir:<id>
open	open	Allocate FD in memory map
read	read	Blob lookup; lazy gen. on miss
write	write	SHA-256 CAS; update metadata
create	create	Atomic Lua via Hypervisor
mkdir	mkdir	Lua create + dot entries
unlink	unlink	ZREM from parent dir
rmdir	rmdir	Empty check + ZREM

‘‘etc’’ to obtain inode 2, then ZSCORE fs:dir:2
‘‘passwd’’ to obtain the target.

C. Layer 3: State Hypervisor

The State Hypervisor abstracts Redis operations into high-level filesystem verbs. Consistency is enforced through server-side Lua scripts that execute atomically within Redis [10].

1) *Redis Data Schema:* Table II shows the key schema optimised for $O(1)$ lookups.

TABLE II
REDIS DATA SCHEMA

Key Pattern	Type	Purpose
fs:inode:<id>	Hash	POSIX metadata (mode, uid, gid, timestamps)
fs:dir:<id>	Sorted Set	Directory entries (score = child inode)
fs:blob:<hash>	String	Content-addressable file data
fs:next_inode	Counter	Monotonic inode allocator

2) *Atomic File Creation:* The atomic_create.lua script executes as a single Redis transaction:

```
-- ARGV: parent_inode, filename, mode, timestamp
local existing = redis.call('ZSCORE',
    'fs:dir:' .. ARGV[1], ARGV[2])
if existing then return -1 end -- EEXIST
local inode = redis.call('INCR', 'fs:next_inode')
redis.call('HSET', 'fs:inode:' .. inode,
    'mode', ARGV[3], 'uid', 0, 'gid', 0,
    'size', 0, 'ctime', ARGV[4],
    'mtime', ARGV[4], 'atime', ARGV[4],
    'nlink', 1)
redis.call('ZADD',
    'fs:dir:' .. ARGV[1], inode, ARGV[2])
return inode
```

Listing 1. Atomic file creation (simplified).

This guarantees that duplicate filenames are rejected, inode allocation is monotonic, and the parent directory entry is linked—all within a single atomic context.

3) *Persistence Layer:* A PostgreSQL persistence layer records all filesystem operations into a file_audit_log table with session correlation, operation type, path, inode, and JSONB metadata, providing a complete forensic trail.

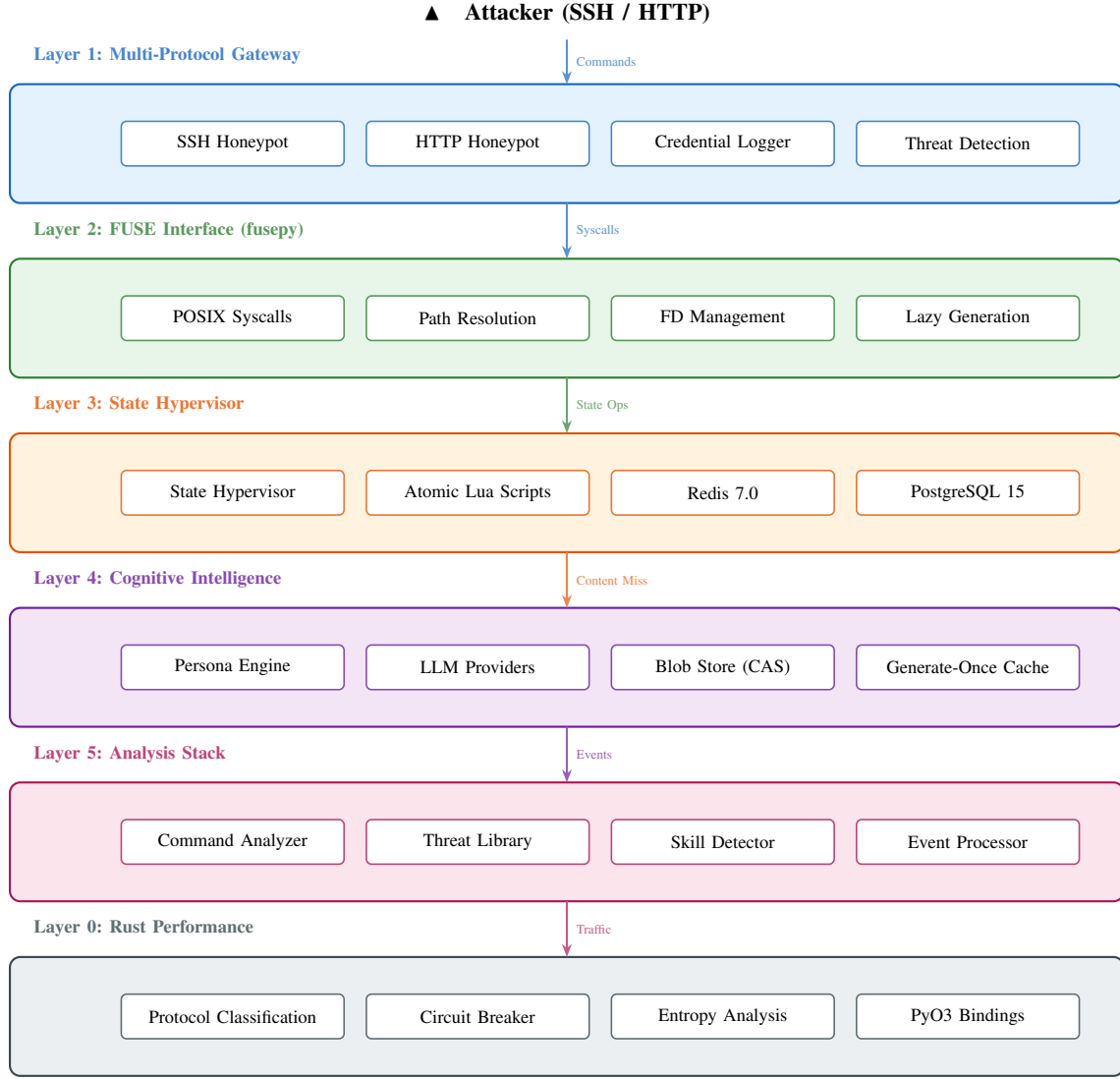


Fig. 1. Chronos six-layer architecture. Data flows downward from the attacker through each layer. State truth resides exclusively in Layer 3 (Redis); LLM content generation in Layer 4 is invoked only on a cache miss and persisted permanently.

D. Layer 4: Cognitive Intelligence

The intelligence layer strictly separates content generation from state management. Fig. 2 illustrates the lazy generation workflow.

LLM Provider Abstraction. An abstract `LLMProvider` base class defines the `generate()` interface. Three concrete implementations exist: `OpenAIProvider` (GPT-4), `AnthropicProvider` (Claude), and `MockProvider` for testing. The active provider is selected at runtime via `LLM_PROVIDER`.

Persona Engine. Switchable personality profiles (e.g., “Ubuntu 22.04 LTS server”, “legacy PostgreSQL database on Debian 10”) are expressed as system prompts. When the FUSE layer encounters a ghost file (no `content_hash`), the Persona Engine constructs a context-aware prompt incorporating filename, path, and active persona.

Generate-Once, Cache-Forever. Generated content is

SHA-256 hashed and stored in `fs:blob:<hash>`. The inode’s `content_hash` is updated atomically. All subsequent reads retrieve content from Redis, *never* from the LLM. This eliminates hallucination entirely and prevents API cost accumulation.

E. Layer 5: Analysis Stack

The analysis stack provides real-time threat intelligence.

Command Analyzer. A regex-based engine detects over 50 attack techniques across eight MITRE ATT&CK tactic categories: Reconnaissance, Persistence, Privilege Escalation, Lateral Movement, Exfiltration, Execution, Defence Evasion, and Credential Access. Each technique contributes a weighted risk score; the aggregate determines a five-level risk classification (benign, low, medium, high, critical).

Threat Library. A curated database of 12 threat signatures with MITRE ATT&CK IDs (e.g., T1059.004 for bash reverse

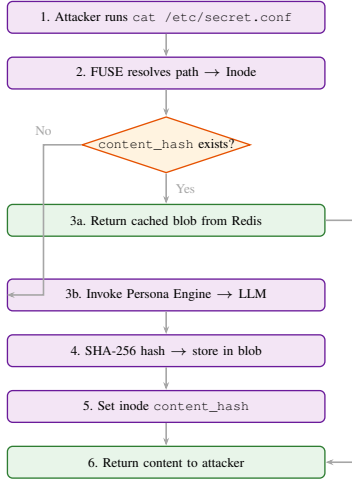


Fig. 2. Lazy content generation workflow. On a cache hit (Yes), content is returned directly from Redis. On a cache miss (No), the Persona Engine generates content via the LLM, which is then stored permanently.

shells, T1548.001 for SUID enumeration, T1505.003 for web shell uploads).

Skill Detector. Classifies attackers across five skill levels—script kiddie, opportunistic, intermediate, advanced, and expert—using eight behavioural indicators including tool sophistication, attack-phase progression, command complexity, evasion technique count, and timing patterns (automated vs. deliberate).

Event Processor and Audit Log Streamer. A sliding-window (default 5 min, 10,000 events) event processor detects patterns for enumeration, privilege escalation, data exfiltration, persistence, and lateral movement. The AuditLogStreamer polls PostgreSQL and distributes events to subscribers via a pub-sub pattern, enabling external SIEM integration.

F. Layer 0: Rust Performance Layer

Performance-critical traffic analysis is implemented in Rust (~62,000 bytes across six source files) and exposed to Python via PyO3 bindings. The layer provides:

- Protocol classification using Aho-Corasick multi-pattern matching;
- Threat detection for SQL injection, XSS, and command injection;
- Circuit breaker with three states (Closed, Open, Half-Open) for load resilience;
- Utility functions: IP validation, Shannon entropy, payload fingerprinting.

IV. IMPLEMENTATION

A. Technology Stack

Table III summarises the technology stack.

B. Data Models

Type-safe interaction is enforced through Pydantic models defining structures for alerts (four severity levels: low–critical),

TABLE III
TECHNOLOGY STACK SUMMARY

Component	Technology
Core Language	Python 3.11
Performance Layer	Rust 1.70+ via PyO3 0.22.0
FUSE Binding	fusepy 3.0.1
Hot State	Redis 7.0 (Alpine, AOF persistence)
Cold Storage	PostgreSQL 15 (Alpine)
SSH Server	Paramiko 3.4.0
Data Validation	Pydantic 2.10.4
Monitoring	Prometheus + Grafana
Containerisation	Docker (CAP_SYS_ADMIN, /dev/fuse)

interaction logs, attacker profiles, SSH command requests, HTTP login requests, and honeypot trigger events. Interaction type is modelled as a four-variant enum (SSH, HTTP, TCP, honeypot).

C. Deployment Architecture

Mirage is deployed via Docker Compose with three services in development (redis-store, db-store, core-engine) and five in production (adding Prometheus and Grafana). The core engine container mounts /dev/fuse and requires CAP_SYS_ADMIN for FUSE operations. Resource limits cap Redis at 512 MB, PostgreSQL at 1 GB, and the core engine at 2 GB. All services communicate over an isolated Docker bridge network (chronos-net).

V. EVALUATION

We evaluate Mirage across four verification phases and two validation suites.

A. Phase 1–2: State and Filesystem Verification

State Hypervisor. Root filesystem initialisation completes with inode 1 (mode 040755). Atomic file creation achieves **8,601 ops/s** (100 files in 0.0116s). Duplicate prevention is verified (FileExistsError on collision).

FUSE Interface. All six core POSIX operations (mkdir, create, write, read, unlink, rmdir) verified. Content round-trip integrity confirmed. Non-empty directory protection raises ENOTEMPTY.

B. Phase 3: Intelligence Verification

Ghost file creation, lazy LLM generation on first read, and content persistence on subsequent reads are all verified. The generate-once policy is confirmed: LLM is called exactly once per unique file.

C. Phase 4: Threat Detection Verification

Table IV shows detection results. The Command Analyzer, Threat Library (12 signatures), and Skill Detector (5 levels) all pass their verification tests.

D. Real Attack Simulation

A validation suite simulates five adversarial scenarios comprising 28 commands spanning reconnaissance, privilege escalation, persistence, credential access, and exfiltration.

Detection rate: 78.6% (22/28 commands correctly classified).

TABLE IV
PHASE 4 DETECTION RESULTS

Test Command	Classification	Risk
ls -la	Benign	0
cat /etc/passwd	Medium	35 (2 techniques)
bash -i >& /dev/tcp/...	Medium	35

E. Comparative Analysis

Table V compares Mirage against traditional and LLM-only honeypots.

TABLE V
COMPARATIVE ANALYSIS WITH EXISTING APPROACHES

Property	Traditional	LLM-Only	Mirage
State Consistency	×	×	✓
Content Realism	Low	High	High
Long-Session Support	×	×	✓
Contradiction Risk	High	High	Zero
Real-Time Analysis	×	×	✓
Scalability	Good	Moderate	Excellent
Per-Session Cost	Low	High	Low

F. Performance Metrics

Fig. 3 visualises the key performance metrics.

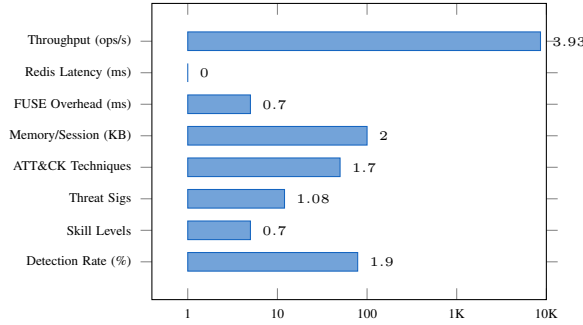


Fig. 3. Key performance metrics across system components (log scale). Throughput is measured in atomic file-creation operations per second.

Fig. 4 shows the processing stage distribution observed across adversarial sessions.

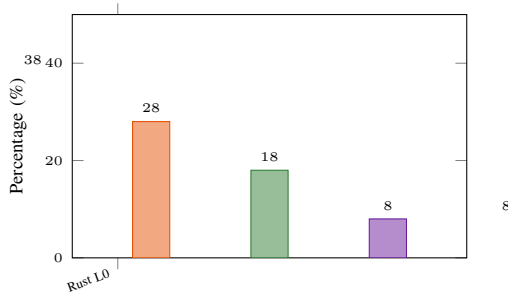


Fig. 4. Processing stage distribution. 66% of requests are handled by the Rust Layer 0 and State Hypervisor without invoking LLM generation, demonstrating the short-circuit routing efficiency.

VI. DISCUSSION

A. Strengths

Mirage’s primary strength is its *architectural* elimination of hallucination. Because the LLM is never consulted for state queries—only for one-time content generation—contradictions cannot arise regardless of session length. This is a structural guarantee, not a probabilistic one.

The FUSE interface provides compatibility with standard Unix tools (`ls`, `cat`, `rm`, `vi`, `gcc`) without modification, supporting realistic interaction without tool-specific emulation.

B. Limitations

Single-host deployment. The current implementation does not simulate network topology or lateral movement across multiple hosts. Multi-host deception requires orchestration of multiple instances.

Protocol coverage. Only SSH and HTTP gateways are currently implemented. FTP, SMTP, RDP, and IoT protocols (MQTT, CoAP) require additional gateway implementations.

SSH-FUSE integration depth. The SSH gateway currently maps a limited set of commands through static handlers; full FUSE-SSH integration routing all commands through the filesystem is a near-term priority.

Dashboard. Threat intelligence is currently accessed through PostgreSQL queries. A web-based real-time dashboard is planned.

C. Future Work

Phase 2 of the project focuses on AI integration under the governing principle that AI must *complement* deterministic behaviour without introducing opaque complexity:

- 1) **AI boundary governance:** enforcing separation between AI-assisted content and deterministic state paths;
- 2) **Explainability:** standardised rationale metadata for detections, surfacing rule-based vs. model-suggested classifications;
- 3) **Quality gates:** regression suites for hallucination detection, consistency checking, and anti-slop filtering;
- 4) **Multi-protocol expansion:** MQTT and CoAP gateways for IoT honeypot scenarios;
- 5) **Automated threshold optimization:** adaptive tuning of short-circuit routing thresholds based on observed traffic patterns.

VII. CONCLUSION

This paper presented Mirage, a state-consistent deception infrastructure that eliminates both the state-inconsistency problem of traditional honeypots and the hallucination problem of LLM-based approaches. By separating state management (Redis with Lua atomics) from content generation (LLMs with generate-once caching) and intercepting filesystem operations at the kernel boundary via FUSE, Chronos provides a structurally sound foundation for high-fidelity deception.

The key insight is that state hallucination in AI-based honeypots is not an inherent limitation of using language models—it is an *architectural* problem arising from the conflation

of state management and content generation. By decoupling these concerns, Mirage demonstrates that LLMs can enhance honeypot realism without compromising consistency.

Evaluation across four verification phases demonstrates atomic state guarantees at 8,601 ops/s, zero-contradiction filesystem operations, and a 78.6% detection rate against realistic adversarial simulations. The framework is deployable via Docker and operates on commodity hardware.

ACKNOWLEDGMENTS

The authors acknowledge the open-source communities behind fusepy, Redis, Paramiko, and PyO3 for providing the foundational components.

REFERENCES

- [1] C. Stoll, “Stalking the wily hacker,” *Commun. ACM*, vol. 31, no. 5, pp. 484–497, 1988.
- [2] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2003.
- [3] N. Provos, “A virtual honeypot framework,” in *Proc. 13th USENIX Security Symp.*, 2003, pp. 1–14.
- [4] M. Oosterhof, “Cowrie SSH/Telnet honeypot,” 2019. [Online]. Available: <https://github.com/cowrie/cowrie>
- [5] J. Zhuge, T. Holz, X. Han, C. Song, and W. Zou, “Collecting autonomous spreading malware using high-interaction honeypots,” in *Proc. 9th ICICS*, 2007, pp. 438–451.
- [6] G. Portokalidis, A. Slowinska, and H. Bos, “Argos: An emulator for fingerprinting zero-day attacks,” in *Proc. ACM EuroSys*, 2006, pp. 15–27.
- [7] S. McKee, “Chatbots as honeypots: Leveraging large language models for deception,” *arXiv preprint*, 2023.
- [8] A. Continella *et al.*, “ShieldFS: A self-healing, ransomware-aware filesystem,” in *Proc. 32nd ACSAC*, 2016, pp. 336–347.
- [9] MITRE, “ATT&CK: Adversarial tactics, techniques, and common knowledge,” 2023. [Online]. Available: <https://attack.mitre.org/>
- [10] Redis Ltd., “Redis Lua scripting,” 2023. [Online]. Available: <https://redis.io/docs/interact/programmability/lua-api/>
- [11] J. Forcier *et al.*, “Paramiko: Python SSH implementation,” 2024. [Online]. Available: <https://www.paramiko.org/>
- [12] T. Bastian, “fusepy: Simple FUSE bindings for Python,” 2023. [Online]. Available: <https://github.com/fusepy/fusepy>